AES-256-CTR Performance Benchmark

A comprehensive benchmark suite for comparing different AES-256-CTR implementations including pure software, AES-NI hardware acceleration, OpenSSL EVP, and Linux kernel AF_ALG.

Features

- **Multiple Implementation Modes:**
  - `soft`: Pure software AES-256 implementation
  - `aesni`: AES-NI hardware accelerated implementation
  - `evp`: OpenSSL EVP (automatic hardware detection)
  - `evp-soft`: OpenSSL EVP (software-only)
  - `afalg`: Linux kernel AF_ALG crypto interface
- **Python Analysis Suite:**
  - Automated benchmarking across multiple data sizes
  - Statistical analysis and visualization
  - Box plots, performance comparisons, heatmaps
  - Export to PDF and PNG formats

Quick Start

1. Build the benchmark

```
make
```

2. Run basic tests

```
# Quick performance comparison
make benchmark

# Test all modes
make test

# Run single mode
./aes --mode evp --size 1048576 --iters 10
```

3. Install Python dependencies and run analysis

```
# Install Python virtual environment and dependencies
make deps
```

```
# Quick analysis (3 sizes, faster)
make quick-analysis

# Comprehensive analysis (8 sizes, more thorough)
make analysis
```

Usage

Command Line Options

```
./aes --mode {soft|aesni|evp|evp-soft|afalg} [options]

Options:
  --mode MODE     Implementation to benchmark
  --size BYTES    Data size in bytes (default: 1048576)
  --iters N       Number of iterations (default: 10)
  --debug N       Debug verbosity level (default: 0)
  --help          Show help message
```

Available Make Targets

```
make all            # Build the benchmark (default)
make test           # Run tests with different modes
make benchmark      # Quick performance comparison
make deps           # Install Python dependencies
make quick-analysis # Quick analysis (3 sizes)
make analysis       # Comprehensive analysis (8 sizes)
make clean          # Remove build artifacts
make help           # Show all available targets
```

Python Analysis

The Python analysis suite generates:

1. **Throughput Box Plots** - Performance distribution across data sizes
2. **Cycles per Byte Plots** - Efficiency comparison
3. **Performance vs Size Charts** - Scaling behavior analysis
4. **Performance Heatmaps** - Relative performance ratios
5. **Summary Statistics** - Detailed statistical analysis

Output Files

- `benchmark_plots/` - Generated visualizations (PNG/PDF)
- `benchmark_results.json` - Raw benchmark data
- `summary_statistics.csv` - Statistical summary

Performance Results (Example)

Typical performance on modern x86_64 hardware:

Mode	Throughput (MB/s)	Cycles/Byte	Description
evp	~7,000	~0.40	OpenSSL optimized
evp-soft	~6,800	~0.41	OpenSSL software
afalg	~3,600	~0.77	Linux kernel
aesni	~1,100	~2.50	Direct AES-NI
soft	~100	~25.93	Pure software

Requirements

Build Dependencies

- GCC with C99 support
- OpenSSL development libraries (`libssl-dev`)
- Linux kernel with AF_ALG support

Analysis Dependencies (Auto-installed)

- Python 3.7+
- matplotlib
- seaborn
- pandas
- numpy

Installation

Ubuntu/Debian

```
# Install build dependencies
sudo apt update
sudo apt install build-essential libssl-dev python3-full python3-venv

# Clone and build
git clone <repository>
cd AES
make deps  # Sets up Python environment
make       # Builds the benchmark
```

Contributing

Feel free to submit issues, feature requests, or pull requests. Areas for improvement:

- Additional cipher modes (GCM, CBC, etc.)

- ARM NEON optimizations
- Windows CryptoAPI support
- GPU acceleration benchmarks

License

This project is released under the MIT License. See LICENSE file for details.

Author

Created by Eduardo Correia ecorreia@apliant.com.br Enhanced with comprehensive analysis by GitHub Copilot Assistant.